FYP SRS Document

Final Year Project

Software Requirement Specification

For

RahScan AI-Powered Pothole Detection & Road Damage Reporting System

(BSCS)

By

Sr#	Name	Registration #/Roll #/Section	Mobile #	E-Mail
1.	Haseeba Rani	Sp-2023/Bscs/142/A	03452776619	haseebarani40@gmail.com
2.	Hafiz Muneeb Ul Hassan	Sp-2023/Bscs/143/A	03114858783	muneebawang@gmail.com

Supervised by:

Ms. Zainab Zafar _____ (Signature)

Department of Computer Science
Lahore Garrison University

Table of Contents

1. Introduction 4
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
2. Overall Description 6
technology departments in Pakistan 6
2.1 Product Perspective 7
2.2 Product Functions 8
2.3 User Classes and Characteristics 9
2.4 Operating Environment 10
2.5 Design and Implementation Constraints 11
2.6 User Documentation 11
2.7 Assumptions and Dependencies 12
3. External Interface Requirements 13
3.1 User Interfaces 13
3.2 Hardware Interfaces 15
3.3 Software Interfaces 16
3.4 Communications Interfaces 17
4. System Features 18
4.1 Citizen Road Damage Reporting Interface 18
4.1.1 Description and Priority 18
4.1.2 Stimulus/Response Sequences 19
4.1.3 Functional Requirements 19
4.2 AI-Based Pothole Detection Engine 21
4.2.1 Description and Priority 21
4.2.2 Stimulus/Response Sequences 21
4.2.3 Functional Requirements 22
4.3 Severity Classification and Confidence Scoring 24
4.3.1 Description and Priority 24

4.3.2 Stimulus/Response Sequences 24
4.3.3 Functional Requirements 24
4.4 Administrative Dashboard and Geographic Heatmap 26
4.4.1 Description and Priority 26
4.4.2 Stimulus/Response Sequences 26
4.4.3 Functional Requirements 27
4.5 Pothole Lifetime Prediction Module 28
4.5.1 Description and Priority 28
4.5.2 Stimulus/Response Sequences 29
4.5.3 Functional Requirements 29
4.6 Predictive Maintenance Module 31
4.6.1 Description and Priority 31
4.6.2 Stimulus/Response Sequences 31
4.6.3 Functional Requirements 32
5. Other Nonfunctional Requirements 33
5.1 Performance Requirements 33
5.2 Safety Requirements 34
5.3 Security Requirements 35
5.4 Software Quality Attributes 36
5.5 Business Rules 37
5.6 Detailed Subsystem Design 38
Reference 38

1. Introduction

Road infrastructure is a fundamental element of economic development, urban mobility and public safety of a nation. The road surfaces in the big cities of Pakistan such as Lahore, Karachi, Rawalpinda etc. are suffering from extensive degradation of their surfaces, which can be attributed to heavy traffic from the vehicles, seasonal monsoon rains, poor construction materials and the lack of proper maintenance. This decay has most significantly manifested itself in the form of potholes which according to the report of Etak and Soomro [11] cause about 30 to 40 percent of traffic accidents in urban areas of developing South Asian countries. Even with this magnitude of the issue, No one for example has any standardized system working in any major city of Pakistan to identify, report, track and proactively manage road damage. There is no organised method for citizens to report bumps, buildings are completely dependent on the manual slow and incomplete inspections and repair teams are only sent after accidents or complaints, and not BEFORE. Distinguished by Salcedo et al. [7] the current road reporting systems are unable to accurately categorize the damage, thus making prioritization of repairs difficult.

1.1 Purpose

This Software Requirements Specification (SRS) document contains all the functional and non-functional requirements for RahScan an AI Powered Pothole Detection and Road Damage Reporting System. RahScan is a complete Web-based Application being developed as a Final year project for Computer Science B.Sc program from Lahore Garrison University. This document outlines the main technical aspects for the development, testing, and evaluation of RahScan, including all major modules of RahScan such as the YOLOv8 based pothole detection engine that was proposed by Jocher et al. [5], the citizen reporting interface, the Pothole Lifetime Prediction module as outlined by Jocher et al. [5] and the Predictive Maintenance module based on the forecasting framework introduced by Taylor and Letham [16], and finally the Administrative dashboard with geographic heatmap. This document's purpose here is to provide a common and clear vision for all stakeholders, including the project supervisor, developers, testers, and academic evaluators, about the required tasks and required behaviors of RahScan.

1.2 Document Conventions

The document is in the form of a standard IEEE SRS document. All functional requirements are labeled with unique identifiers in the format REQ-X for easy traceability. There is a numeric hierarchy of

headings. In this document, the term "system" is used to denote RahScan as a whole. If a citizen, administrator or supervisor is using the system, they are considered a user. The following "detection model" is the object detection model YOLOv8n from Jocher et al. [5] which has been trained with the dataset RDD2022 from Arya et al. [6]. Where appropriate, the requirements listed as High Priority are identified as such. The IEEE Author-Date hybrid style of referencing, where the name of the author is mentioned in the text and the reference is cited in square brackets, is followed in this document.

1.3 Intended Audience and Reading Suggestions

The following readers will find this SRS helpful:

- **Project Supervisor (Ms. Zainab Zafar):** Reviewed system requirements, analyzed technical scope, and gave academic guidance. Starting from Section 1 (High Level Requirements) and Section 2 (Requirements), followed by the detailed requirements in Sections 3, 4 and 5 is recommended.
- **Developers:** (Haseeba Rani and Hafiz Muneeb Ul Hassan): to provide guidance for design, development and integration for all RahScan modules. All sections apply, and some special suggestions are made for Sections 3, 4 and 5 for implementation.
- **Testers:** to create test cases and to ensure that the system satisfies all requirements. For Test Case Design needs sections 4 (System Features) and 5 (Non-Functional Requirements) are the most relevant ones.
- **Examiners and Evaluators:** to evaluate the system specification on completeness, clarity and correctness. It is recommended to read the sections 1-5 in this sequence to gain a complete understanding of RahScan.

1.4 Product Scope

RahScan is designed as an AI-based, web-based road damage detection and reporting application for the urban road network in Pakistan. Heavy traffic, the monsoon season and poor maintenance practices significantly contribute to the deterioration of Pakistan's roads. It has been documented by Etak and Soomro [11] that the roads surfaces in South Asian developing countries have been found to be one of the major cause of urban traffic accidents anywhere that accounts for about 30-40 percent which is more substantial in high traffic areas of Lahore and Karachi in Pakistan. In developing world urban areas, Karballaeezadeh et al. [15] also showed that the costs of infrastructure in the absence of intelligent

inspection and data-driven maintenance planning are considerably higher and the number of accidents avoidable. In the current scenario in any major city in Pakistan there is no centralised, data-dependent system to systematically identify, report and plan for action in repairing potholes, and RahScan is a first-of-its-kind solution within this context.

Citizens can report road damage by uploading photos via RahScan's easy-to-use Web interface. Each uploaded image is passed through a YOLO v8 object detection model implemented by Jocher et al. [5] that was trained on a dataset by Arya et al. called RDD2022 with more than 47,000 annotated images of road damages from six countries and then fine-tuned on locally collected road images from Pakistan. The approach followed by RahScan is also confirmed by other studies, as shown by Asad et al. [1] which demonstrated that deep learning models can be easily applied to standard cameras to achieve robust pothole detection performance without the need of specific sensor-based hardware. The system automatically identifies potholes, categorizes their size as Small, Medium or Large, and provides the citizen with marked up results. All findings are displayed on a dashboard using an interactive geographic heat map (Leaflet.js), filterable tables and live statistical data to be shown to government administrators. In addition to, or with, the detection algorithm, RahScan features a Pothole Lifetime Prediction module that relies on a regression-based model developed with Scikit-learn Random Forest Regression as described by Pedregosa et al. [18] and live weather data from the OpenWeatherMap API to estimate the rate of deterioration for each pothole detected, an approach that is similar to what Marcelino et al. [13] confirmed: regression-based models are especially well suited for the prediction of pavement deterioration rates based on environmental and traffic data. A Predictive Maintenance module based on the Facebook Prophet forecasting framework proposed by Taylor and Letham [16] predicts maintenance needs in the next 30 days, giving a budget estimate, by analyzing the past records of road damages. RahScan is aimed at transitioning a slow and reactive, complaint driven road maintenance from Pakistan to an intelligent, data-driven, fast system that not only saves lives but can also lower the cost of road maintenance and enhance urban infrastructure management.

2. Overall Description

RahScan is a new, self contained web application that has a gap in the market in Pakistan which no existing platform has addressed. Salcedo et al. [7] noted that there is no existing system that provides a seamlessly operationalized reporting process that can be integrated into citizen-friendly reporting, automated severity

classification, and administrative decisions. RahScan tackles this challenge by putting citizens, AI detection engine, and government administrators on a single platform, with three interdependent layers. The first layer is a citizen interface website for submitting the photos and location of the road damage. The second layer is an AI processing backend that processes all uploaded images in real time, classify them based on the severity of the pothole, and generate deterioration predictions based on the severity of the pothole using a Random Forest Regression model from the Scikit-learn library Java library implementation of which is done by Pedregosa et al. [18], and the model was trained on the RDD2022 dataset published by Arya et al. [6] using a YOLOv8 detection model as implemented by Jocher et al. [5]. The third layer is an administration dashboard featuring an interactive Leaflet.js geographic heatmap with which government officials can keep track of and take action on all reported road damage across Lahore, Karachi and Rawalpindi. A central coordinating engine, based on FastAPI's RESTful API, connects all three. Instead, RahScan is a standalone deployable web application and does not rely on any existing government IT infrastructure so that it can be practically implemented in the resource constraints of municipal technology departments in Pakistan.

2.1 Product Perspective

RahScan is an entirely new, self-contained web-app solution, rather than replacing any current government system, but filling an important gap that none of the existing systems would do. Salcedo et al. [7] were right in their review of existing road maintenance platforms when they noted that no platform has been found to be able to achieve an integration of citizen centric reporting and automated severity classification and an operational decision support system. RahScan is made to fill this gap. There is no central road damage detection and reporting system in any major city of Pakistan and RahScan is the first complete AI-based application that connects citizens, AI detection engine and government administrators in one platform.

RahScan comprises three key interrelated layers. The first is the online application that is submitted by citizens, with information about the damage to the road and its location.The first is a citizen-oriented web interface for submitting information about damage to the road and its location. The second is a pothole-image analysis system based on YOLOv8 to detect potholes on their images and tree deterioration prediction system based on the Random Forest Regression model of the Scikit-learn library, which processes all the images uploaded in real time, classifies the severity of the pothole, predicts the

deterioration of the trees, and saves all the results to a PostgreSQL database. The third one is a dashboard for administration, which provided interactive Leaflet.js geographic heatmap to monitor, manage and take proper action on all reported damage on roads in Lahore, Karachi or Rawalpindi. The three layers are connected via a RESTful back-end implemented using FastAPI as the central control plane of the whole system. Not only it does not require integration with any existing government IT infrastructure, but it is also a deployable web application, which can be deployed within the resource constraint of a municipal without any hassle.RahScan is essentially a deployable web application and does not need to be integrated with any existing government IT infrastructure, which makes it practically deployable within the resource constraint of a municipal without any hassle of technology departments.

2.2 Product Functions

RahScan offers the following key feature:

- **Citizen Road Damage Reporting:** Citizens can submit a picture of road damage that they've taken and the location can be submitted via a simple web application with no technical experience or special hardware other than a basic smartphone and/or computer.
- **AI-Based Pothole Detection:** All images submitted are instantly classified by a YOLOv8 model as implemented in Jocher et al. [5] and trained on a published pothole dataset titled RDD2022 [6] and supplemented with locally collected Pakistani road images. As argued in Asad et al. [1] such models can be used to perform good detection with standard cameras without any of the specific sensor hardware. Model identifies potholes, creates bounding boxes and provides tagged return results to the citizen in real-time.
- **Severity Classification:** Following the rule-based approach proven by Thakur et al. [3] and Bhavana et al. [4] for their YOLOv8-based road damage detection systems, each pothole detected is automatically assigned a severity label based on the area of the corresponding bounding box relative to the full image frame.
- **Confidence Scoring**: Each detection produces a confidence score which provides a level of confidence for the each pothole's report to the administrator.
- **Report Storage and Management:** All detection results, their severity, their confidence as well as their location and date are stored into a structured PostgreSQL database and are managed by a repair workflow: Pending - In-Progress - Resolved

- **Geographic Heatmap Visualization:** On admin dashboard, it shows and highlights the spatial distribution and severity concentration of all the reported potholes across Lahore, Karachi and Rawalpindi in an interactive geographic heatmap represented in Leaflet.js libraries.
- **Pothole Lifetime Prediction:** Based on Pedregosa et al. [18] work, Pothole Lifetime Prediction models the lifetime of a detected pothole using a combination of current severity based on visualization, road traffic density, and real-time weather forecasts from the OpenWeatherMap API. This has also been substantiated by Etak and Soomro [11] who found temperature, rain and traffic volume as the most important contributors to worsening pavement deterioration.
- **Predictive Maintenance Forecasting:** This machine-learning project [16] [17] introduces a time-series forecasting module in Facebook Prophet that, based on past road damage data, can predict failure patterns for road sections more prone to damage, creating proactive maintenance plans up to 30 days ahead which show an estimated repair budget for each section.
- **Admin Report Filtering and Assignment:** Administrators can filter and sort on reports by city, severity, date range, and repair status and they are also able to assign a repair team and change the status of a report straight from the dashboard.
- **Data Quality Flagging**: To assist in the identification of unreliable data submissions and high-risk zones, data quality management approach recommended by Chengula et al. [8] was implemented by automatically flagging data quality reports with incomplete location data and detection confidence scores below 0.50.

2.3 User Classes and Characteristics

RahScan supports 3 different types of users:

- **Citizens**: General public users who notice damming or wear on the road in their everyday driving. For a citizen-to-citizen road damage reporting platform to be viable practically, it must be simple, and approachable to all citizens [7][9]. Consumers of RahScan are thus expected to lack technical expertise, and their interface with the system is restricted to uploading a road photo, typing a location, submitting the report, and viewing the detection result (annotated on the photo). They can be used with a cell phone or computer and it doesn't require internet connection.
- **Administrative Users (Government Authorities)**: This is for municipal authorities, road maintenance departments, and other traffic management officers who use the admin dashboard to

track reported potholes, view the geographical heatmap, manage the process for repairs, review lifetime predictions and maintenance forecasts, and assign a maintenance team. Basic computer skills and knowledge of web-based dashboards is expected for these users.

- **System Supervisors and Project Evaluators:** Academic supervisors/ University examiners to check the correctness, performance, and completeness of the system. These users have the opportunity to get access to both the citizen interface and admin dashboard for evaluation and are expected to have a background of software engineering and AI.

2.4 Operating Environment

RahScan will run in the following environment:

- **Client Side:** Any modern web browser including Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari running on Windows, macOS, Linux, Android, or iOS devices. No browser plugins or additional software are required.
- **Server Side:** On the backend it is required to be a standard Linux server running Ubuntu 22.04 or above and Python 3.10 or above. The backend API and the inference pipeline of YOLOv8 will be provided via FastAPI. The relational database is going to be PostgreSQL 14+ that handles all the relationships.The relational database will be used for all relationships is going to be PostgreSQL 14 or higher.
- **AI Model Training Environment:** Based on the recommendation of Jocher et al. [5] to build reproducible deep learning models without institutional computing infrastructure, we will use the open access GPU services on Google Colab for AI model training. The value of the model weights will be exported and will be applied to the production backend.
- **External API Dependency:** The Pothole Lifetime Prediction module needs internet connectivity to receive live weather forecasts from the OpenWeatherMap API that provides the rainfall probability and temperature as the two most essential environmental features required by Etak and Soomro [11] to predict deterioration of potholes.
- **Network:** To make citizen image uploads, access to admin dashboard, and calls to the weather API, Network RahScan needs a stable internet connection. The system is designed to operate with the normal broadband internet that is being provided in urban areas of Pakistan.

2.5 Design and Implementation Constraints

While using RahScan, the following limitations apply:

- **Hardware Limitations:** Suitability for Hardware RahScan needs to be able to be deployed on standard server hardware and not require a high-end GPU infrastructure to run, as there are limited amounts of computing power available at the Pakistani municipal departments. For this reason, the relatively small model variant YOLOv8n has been chosen specifically because, as described by Jocher et al. [5], it can be used for real-time inference on CPU or low-end GPU hardware and has a competitive detection accuracy.
- **Dataset Availability:** Data on road damage in the cities of Pakistan is not available publicly. Matarira et al. [14] recognized that most predictive maintenance models used in developing countries should rely first on simulated or small amounts of historical data and eventually switch to real data as historial data becomes available. The Predictive Maintenance module will be the same.
- **Local Road Data:** The images for fine-tuning the detection model need to be collected manually from Lahore and Karachi as the images in RDD2022 [6] published for hand consideration are not available for Pakistani cities.
- **Technology Stack:** The system has to be implemented using Python for the backend and the AI model (YOLOv8), React.js for the frontend, PostgreSQL for the database, and Leaflet.js for the geographic heatmap as per project proposal approved by the supervisor.
- **Budget Constraints:** Budget Constraints All of the tools, libraries and services utilized must be free or open-source. Weather data will be sourced from the OpenWeatherMap API, freely available, open source library of models and algorithms will be used for model training, and Scikit-learn library, as stated by Pedregosa et al. [18] (2011), and the Facebook Prophet library, as introduced by Taylor and Letham [16] (2017), are also freely available open source libraries.
- **Academic Timeline**: The entire system needs to be built, tested, and showcased in a six month period as defined by the project milestones and outlined in academic timeline.

2.6 User Documentation

The following documentation will be provided with RahScan:

- **User Manual:** An easy step-by-step guide for citizens to open the web interface, upload a photo of a road damage, enter a location, submit a report and read the detection result. In plain, non-technical language suitable for general public users as suggested by Li et al. [9] for crowdsourced reporting platforms of road damage.
- **Admin Guide:** An explanation of how to log in to the dashboard, view and filter reports, understand the heatmap, read the outputs of the lifetime prediction system, maintenance forecast system, update the repair statuses and assign repair teams.
- **Technical Documentation**: Technical Documentation API documentation for all RESTful endpoints, request and response formats, DB model and schema, details on integration etc. Intended for developers and evaluators.
- **Project Report:** A formal final year project report, including system design and methodology, implementation, and results of testing, with evaluation metrics, required as part of one's academic requirements at Lahore Garrison University.

2.7 Assumptions and Dependencies

RahScan assumes the following:

- The citizens are assumed to possess a smartphone / computer with camera and internet connection that is able to upload the images of the road, which are similar to the citizen reporting model used and validated by Salcedo et al. [7] and Li et al. [9].
- It is assumed that the free tier of the OpenWeatherMap API will stay available and will provide accurate weather forecast information for all the cities in Pakistan throughout the life cycle of the project. According to Etak and Soomro [11] the most important inputs to the Pothole Lifetime Prediction module are weather data, confirming that rainfall probability and temperature are the two most important weather parameters for the deterioration of potholes.
- Following the assumptions made by Jocher et al. [5] and recommended by the reproducible deep learning experimentation, it is assumed that Google Colab will continue to be able to supply free GPU resources for the duration of the project which are sufficient to train the YOLOv8 model.
- It is assumed that the RDD2022 dataset published by Arya et al. [6] will be publicly accessible for download from the repository and will be used as primary training dataset

- It is assumed that 500 to 700 road images of Pakistan collected locally are enough to fine-tune the YOLOv8 model for local road conditions, in the hope of good evaluation results.
- RahScan relies on the Ultralytics YOLOv8 library as released by Jocher et al. [5], Scikit-learn as described by Pedregosa et al. [18], Facebook Prophet as introduced by Taylor and Letham [16] as well as FastAPI, PostgreSQL, React.js, and Leaflet.js. Adjustments may need to be made to implementation due to any breaking changes in these libraries during the development period.
- Based on the approach used by Matarira et al. [14] in early stage predictive maintenance systems having limited historical data, it is assumed that the synthesized historical damage data generated for the Predictive Maintenance module will be representative enough to show the capability of the module during the academic evaluation period.

3. External Interface Requirements

3.1 User Interfaces

RahScan offers two different Web-based user interfaces, one for citizens and one for administrative users. Both interfaces will be built with React and will be able to be accessed with any standard modern browser without any installation or plugins. As stated by Li et al. [9] in their research on crowdsourced road damage reporting systems, the practical implementation of any road damage reporting system targeted at citizens is solely reliant on the ease and accessibility of its user interface, which has directly influenced the design of RahScan's citizen user interface.

Citizen Web Interface:

The citizen interface has been made as user friendly and simple as possible without any technical knowledge required by the user. It will have the following screens and components:

- **Home Page**: A clean page that explains what RahScan is, how it works and a clear button for "Report a Pothole" that will lead citizens to the reporting form.
- **Report Submission Form:** A simple form with an image upload button to select the road damage images, a textbox to enter the road location, and a Submit button. The form will show the loading indicator during the processing of the image by the AI detection engine.

- **Detection Result Screen**: On successful submission, the citizen would see the uploaded image returned with the bounding boxes drawn around all the detected potholes, a severity label for each detected pothole (Small/Medium/Large) according to the method validated by Thakur et al. [3] and a confirmation message that the report has been submitted.
- **Error Messages:** Clear, plain-language error messages will be displayed if: The image format is not supported; The location field is empty; The server is temporarily unavailable. All buttons and form elements will be in the same design and style (minimal). The font will be large enough for mobile use. The interface will be fully responsive, and will display properly on every screen size from mobile to desktop.

Administrative Dashboard Interface:

The admin dashboard will be the main control panel for government officials will have the following features:

- **Login Screen:** A secure login page with username and password fields to allow only allowed dashboard users to access the dashboard.
- **Overview Panel**: A summary statistics panel at the top of the dashboard that presents the following information: total reports received, total reports by severity level, current resolution rate, and weekly and monthly trend comparisons.
- **Interactive Geographic Heatmap:** A full width map (Leaflet.js) centered on Pakistan with the density and severity distribution of all reported potholes in Lahore, Karachi and Rawalpindi. Red zones are areas of clusters with high density and severity, green zones are areas of low or no active reports. The Map will provide zoom, pan and click-to-view-report-details interactions.
- **Report Management Table:** A table that displays all the reports submitted and can be filtered and sorted by columns including Report ID, Location, Severity, Confidence Score, Submission Date, Lifetime Prediction, and Current Status. One of the most significant weaknesses of existing road complaint systems, as documented by Chengula et al. [8] is the lack of filtering and prioritization capabilities to help identify complaints.Chengula et al. [8] documented that one of the most important weaknesses of the existing road complaint systems is not having the capacity to filter and prioritize complaints. This is exactly where RahScan comes in.

- **Report Detail View:** Clicking on any report in the table will open up the report detail form with the following content: original uploaded image with detection annotations, all detection outputs, lifetime prediction output with urgency reason generated from the Random Forest model mentioned by Pedregosa et al. [18], and current repair status with a possibility to update the status.
- **Repair Status Workflow:** There are three status stages within a repair report: Pending - In Progress - Resolved. Admins can assign repair teams to a status and update the status directly from the report detail view.
- **Predictive Maintenance Forecast Panel**: A dedicated panel showing the 30-day forecast of maintenance performed by the Facebook Prophet module developed by Taylor and Letham [16] that includes road segments grouped as Urgent, Plan Repair and Monitor, and an estimated budget for the maintenance during the forecast period.

3.2 Hardware Interfaces

RahScan is software for the Web and doesn't actually connect with the physical hardware components. Other hardware interactions, however, relate to the operation of the system:

- **Camera and Image Capture Device:** Citizens will take pictures of the damage to the road using the camera of their smartphone, tablet or computer. Recently, Asad et al. [1] have validated that the image quality of a standard smartphone can be used to reliably detect potholes using deep learning models in real-time. RahScan does not directly control or communicate with the camera hardware – the image files are simply selected and uploaded via the standard file select dialog of the browser.
- **Server Hardware**: The RahScan backend, database, and AI inference pipeline will be deployed on a typical Linux server. YOLOv8n is chosen as the model variant to achieve real-time performance on standard CPU hardware, which is suitable for the production deployment of RahScan as noted by Jocher et al. [5] without the need of expensive computing infrastructure to run the model.
- **Network Interface**: To enable the communication of citizen interface with the back end API and the administrative dashboard via HTTP, the system needs a working network interface on both the client device and the server.

- **Storage**: Storage space on the server is needed to accommodate uploaded road damage images, the PostgreSQL database and the YOLOv8 model weights. It is recommended to have at least 20 gb of server storage available for initial deployment.

3.3 Software Interfaces

The following external software components, libraries and services are integrated with RahScan:

- **YOLOv8 Ultralytics Library (v8.x):** YOLOv8 Ultralytics Library (v8.x) YOLOv8 is the key object detection backbon of RahScan as released by Jocher et al. [5]. These weights are loaded in the YOLOv8n model during the startup of the FastAPI backend, and are called on each image submission. The input is a road damage image, and the output is a list of the bounding boxes, class labels and confidence scores.
- **Roboflow:** During the dataset preparation phase, image annotation, version control of the dataset and applying augmentation techniques was done using Roboflow, which was done on RDD2022 dataset published by Arya et al. [6] along with locally collected Pakistani road images. Once the annotated dataset has been exported as a Roboflow set, it can be used for training in Google Colab using YOLOv8.
- **PostgreSQL (v14 or higher)**: Primary relational database for storing all road damage reports, detection outputs, location, severity classification, confidence score, repair status, and historical road damage records used by the Predictive Maintenance module is PostgreSQL (v14 or higher).
- **Scikit-learn:** The Pothole Lifetime Prediction module uses the Random Forest Regression model trained and executed using Scikit-learn [18]. The input parameters are: pothole severity, Bounding Box area, Traffic density level, weather parameters confirmed by Etak and Soomro [11] as most important pothole deterioration predictors and seasonal indicators. Output is a forecasted number of days until the pothole is critical.
- **Facebook Prophet**: Facebook Prophet, as introduced by Taylor and Letham [16] and confirmed to be suitable for infrastructure management applications by Daraghmeh et al. [17], is employed in the Predictive Maintenance module to undertake time-series forecasting on the historical road damage records per location. The output is a forward prediction of the damage recurrence over the next 30 days, while the input consists of a time series of damage history for a road section.

- **OpenWeatherMap API (Free Tier)**: API used by Pothole Lifetime Prediction module to access live weather forecast information such as rainfall possibility, temperature etc. at the pothole location. Etak and Soomro [11] and Fakhri and Dezfoulian [12] confirmed that weather variables are one of the most important predictors of accelerated pavement deterioration, so integrating this API is an important part of the prediction pipeline.
- **Leaflet.js**: Leaflet.js has been integrated within the React.js admin dashboard to render the interactive geographic heatmap that shows the distribution and severity of potholes across Lahore, Karachi and Rawalpindi. Leaflet.js is free of charge and open source.
- **FastAPI (Python)**: The RahScan backend was developed using FastAPI (Python), which was used as the web framework to provide RESTful API endpoints for submitting images, performing AI detection inferences, retrieving reports, updating a status, delivering heatmap data, and serving predictions results.
- **React.js**: Single page web applications were created using React.js both for the citizen interface and for the administrative dashboard.
- **Google Colab**: It was only used during the training phase of the model, to train the YOLOv8n and YOLOv8s variants on the prepared dataset, as recommended by Jocher et al. [5] to perform deep learning experiments without institutional computing infrastructure.

3.4 Communications Interfaces

RahScan takes advantage of the following communication protocols and interfaces:

- **HTTP and HTTPS**: All communications between citizen web interface, Admin dashboard and FastAPI backend are done through HTTP during development and HTTPS in the production deployment. All client-server communication is set up in a standard RESTful API way of communication.
- **REST API**: The FastAPI backend provides RESTful endpoints to take and deliver JSON formatted data. The following API endpoints will be created:
    - **POST /upload** Takes an image of road damage and string of location from the citizen interface, runs YOLOv8 inference (as done by Jocher et al. [5]), saves the report to the database, and returns annotated results of the detection.

- **GET /reports** Returns a list of all Reports stored, optionally filtered by city, severity, date range and status.
- **GET /reports/{id}** Returns detailed information of a single report (including the results of the detection, lifetime prediction by the Scikit-learn Random Forest model described by Pedregosa et al.[18] and the current status of that report).
- **PUT /reports/{id}/status** Update the repair status of a specific report, as an admin user.
- **GET /heatmap** Returns Geographic coordinates and severity data of all the current reports to populate the Leaflet.js heatmap.
- **GET /maintenance-forecast** Facebook Prophet forecasting predictive maintenance for a 30-day window for all road segments along with budget estimates.

- **OpenWeatherMap API (HTTPS):** RahScan makes HTTP (HTTPS) GET requests to the OpenWeatherMap API for the latest weather forecast for a specified city or location. The validations of the approach of deterioration prediction based on weather conducted by Etak and Soomro [11] and Fakhri and Dezfoulian [12] is supported by the response, which is reported back as a critical feature to the Pothole Lifetime Prediction module.
- **Browser-Server Communication**: The React.js front end is connected to the FastAPI back end using the regular Fetch API or Axios library, and data is sent as a JSON payload and multipart form data for uploading images. The API returns all the responses in JSON format.
- **Database Connection**: The FastAPI backend interacts with the PostgreSQL database over a local TCP connection and connection with postgres database is done in Python, which uses a database driver. The database doesn't point to the web and it is just accessible from the backend server.

4. System Features

4.1 Citizen Road Damage Reporting Interface

4.1.1 Description and Priority

This feature enables citizens to report road damage by uploading a photo and specifying a location, using a simple, accessible website. Both Salcedo et al. [7] and Li et al. [9] highlighted in their papers the importance of having no barriers to participation for any citizen reporting system, and RahScan is no

exception with no need to register accounts, no technical skills required, and no special hardware. This is the access to the whole RahScan pipeline and is considered to be of **High Priority**.

4.1.2 Stimulus/Response Sequences

- Citizen accesses RahScan Web App in any browser - System shows the home page which contains a Report a Pothole button.
- Citizen enters in an address and uploads a photo of the pothole - System submits the pothle form to the database.
- Citizen chooses a road damage picture from their device - System displays the chosen picture on the screen.
- Citizen types in road location in the text box - System accepts and stores the input.
- Citizen clicks 'Submit' - System shows a loading indicator as the image is sent to the backend for processing.
- Backend successfully processes the image - System returns the annotated image with bounding boxes, severity labels, pothole count, and a success confirmation message to the citizen.
- No location is entered in the image, when it is submitted, it shows an error message: "Please enter the road location before submitting.
- A file format other than an image (JPG, PNG) is uploaded - An error message appears on the system: "Please upload a valid image file (JPG, PNG).
- Server is temporarily unavailable - System shows an error message: "Something went wrong. Please try again."

4.1.3 Functional Requirements

- **REQ-1:** An installation-free, non-login report submission form shall be provided via the web for accessing from any modern browser, on any device.
- **REQ-2:** System should enable the citizen to upload the image of a damaged road in JPG/PNG format, not to exceed 10MB.
- **REQ-3:** It will show a real time preview of the selected image when submitted.
- **REQ-4:** If the location field is not filled on the form before it is submitted, the system shall show an appropriate error message.

- **REQ-5:** The system should show a loading screen when uploading an image is in progress and the AI detection engine is active.
- **REQ-6:** When the citizen submits the image for detection, the annotated detection result image should be returned to the citizen within 5 seconds, when normal servers load.
- **REQ-7:** The number of potholes detected, the level of severity of each pothole detection and confirmation message that the system displays when the report is submitted shall be on the result screen.
- **REQ-8:** The system should present error messages in plain language for any inputs and failings which are not valid.

Use Case Diagram:

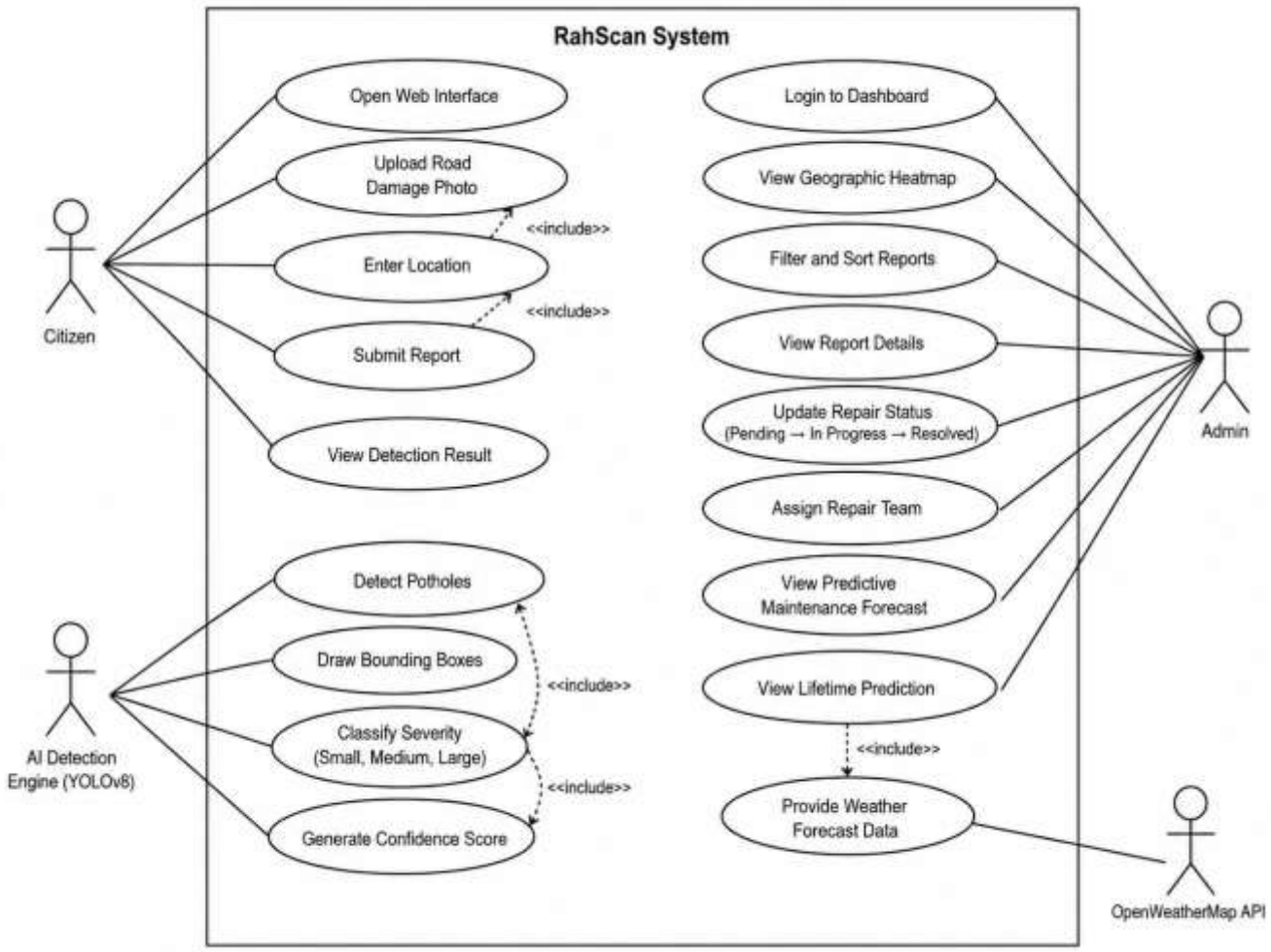


Figure 4.1: Use Case Diagram of RahScan System

4.2 AI-Based Pothole Detection Engine

4.2.1 Description and Priority

This characteristic is the very heart of RahScan. It runs all submitted road damage images in real-time, utilizing a YOLOv8 object detection model implemented by Jocher et al. [5] which is pre-trained on a road damage dataset, RDD2022, from Arya et al. [6] with more than 47,000 annotated road damage images from six countries. As shown by Asad et al. [1] deep learning models can be used to detect potholes from a standard camera input, without the need to add specialized sensor hardware. Thakur et al. [3] and Bhavana et al. [4] investigated YOLOv8 based architectures further, which verified that YOLO can detect road damage (in hard light conditions) and support this reasoning of model selection of YOLOv8 for RahScan. This feature is of **High Priority**.

4.2.2 Stimulus/Response Sequences

- A citizen interface sends to the backend a road damage image - The system loads the image into the road damage inference pipeline designed by Jocher et al. [5] by the YOLOv8.
- YOLOv8 model processes the image - System identifies all pothole instances in the image and generates bouding box coordinates, class labels and confidence scores for all the pothole detections.
- The presence of one or more potholes is identified - The system highlights the images with the bounding boxes of the detected potholes, then forwards the detection results to the severity classification layer and stores all these detection results in the database.
- No potholes are detected - System returns the original image with a message: "No potholes detected in the submitted image."
- Low resolution or blurred image for reliable detection – Detection results are returned at low scores which are automatically marked in the database for further data quality review.

4.2.3 Functional Requirements

- **REQ-9:** Use a YOLOv8n model as described by Jocher et al. [5] as the basic detection network for real time pothole detection..
- **REQ-10:** The detection model is trained on the RDD2022 dataset by Arya et al. [6] which consists of road damage images totaling more than 47,000 images, with these images also divided through the addition of 500 to 700 of locally captured road images from Lahore and Karachi.
- **REQ-11:** On a single inference, the system is able to detect multiple pothole instances in a single submitted image.
- **REQ-12:** Bounding boxes shall be drawn around each image of a pothole detected by the system in the output image.
- **REQ-13:** System shall output a conf score in [0,1] for any instance of pothole found.
- **REQ-14:** mAP@0.5 shall be ≥ 80 on Pakistani road image test set held out.
- **REQ-15:** Under standard server hardware environment, the system shall finish the inference of pothole detection for each image in less than 3 seconds.
- **REQ-16:** If the potholes are not detected at any time and a response is returned to the citizen interface, then no exception shall be thrown by the system.

Activity Diagram:

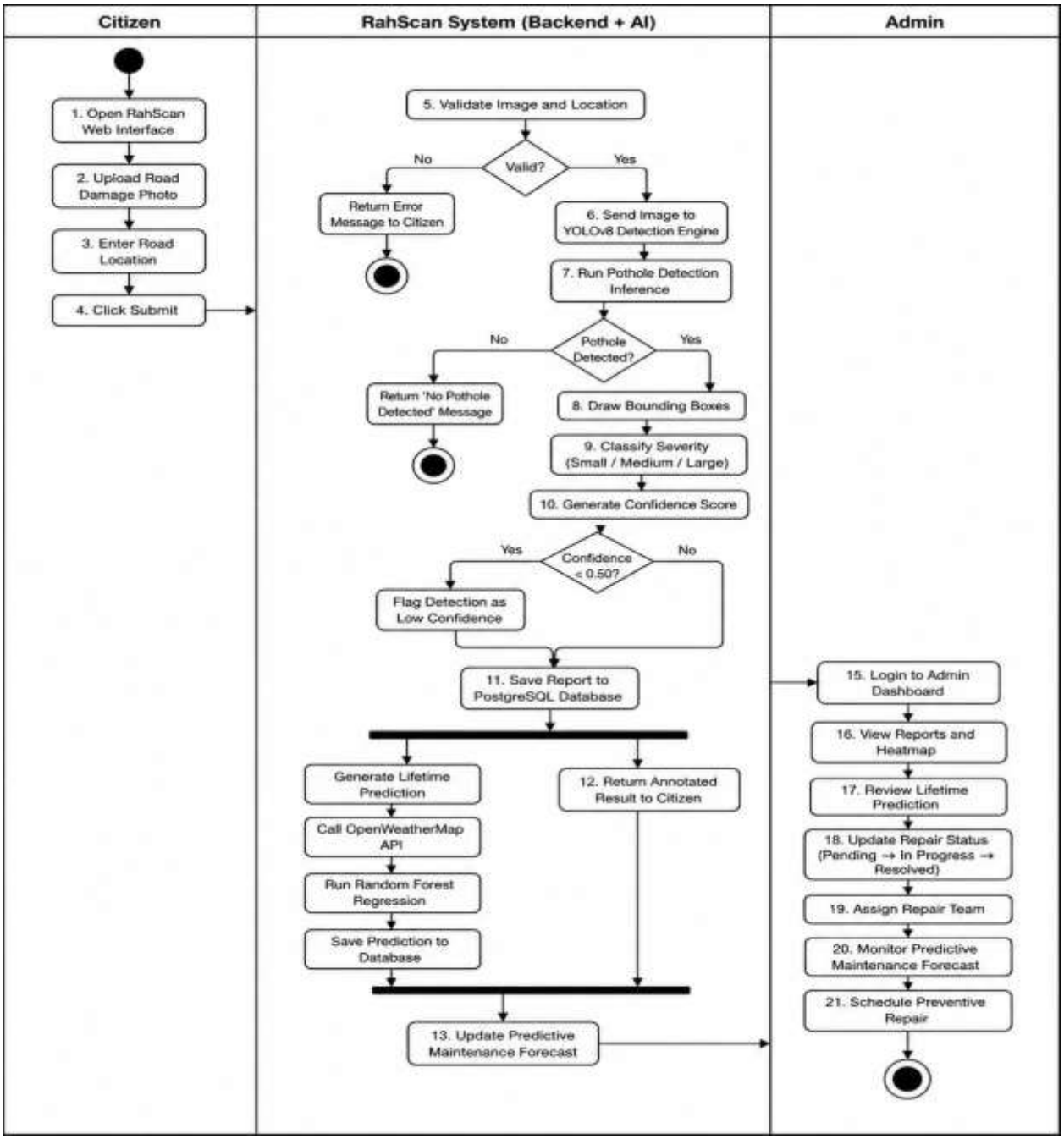


Figure 4.2: Activity Diagram of RahScan System

4.3 Severity Classification and Confidence Scoring

4.3.1 Description and Priority

This automatically sorts the severity of each pothole detected into one of three categories: Small, Medium and Large depending on the percentage of area of the bounding box to the image's full frame. In this is one of their most serious limitations, pointed out by Salcedo et al. [7]: there is a lack of consistency among existing road reporting platforms in rating their level of damage; severity data cannot be used to determine and prioritize repair orders otherwise. RahScan fills this void by providing a lightweight rule-based severity classification layer that can be executed on the output of the bounding boxes from the YOLOv8 model as it has been implemented by Jocher et al. [5] without the need of a separate classification network. High Priority - This feature is of High Priority.

4.3.2 Stimulus/Response Sequences

- YOLOv8 detection engine receives bounding box co-ordinates of the detected pothole and the system calculates the area of the bounding box and compares it with the entire image frame - area percentage is calculated.
- Less than 5% was calculated - System identifies the pothole as being classified as a Small pothole.
- If the percentage for calculated area falls between 5 and 15 percent - System categorizes the pothole as Medium.
- >50% area calculated - System automatically classifies it as Large and is given immediate attention.
- Classification is complete - Now the system stores the confidence score, severity label, coordinates of the bounding box and other information about the report into the database.
- System displays the Pothole severity and confidence score in the report detail view when Admin views a report.

4.3.3 Functional Requirements

- **REQ-17:** The system should use 3 severity levels – Small, Medium and Large, determined by the ratio of the area inside the detected bounding box of each pothole to the area of the full image.

- **REQ-18:** The system must detect potholes that are less than 5 percent of the image and give them the classification " is Small.
- **REQ-19:** If there are potholes between 5% and 15% of the image, it should be classified as Medium.
- **REQ-20:** The system should be able to identify a pothole that exceeds 15% of the image as a Large pothole and automatically set a flag for immediate repair of the pothole.
- **REQ-21:** Store the YOLOv8's confidence score for each detection in addition to severity classification into the database.
- **REQ-22:** System shall be able to show the severity (label) and confidence score for each identified pothole on the citizen result screen as well as on the admin report detail view.

State Machine Diagram:

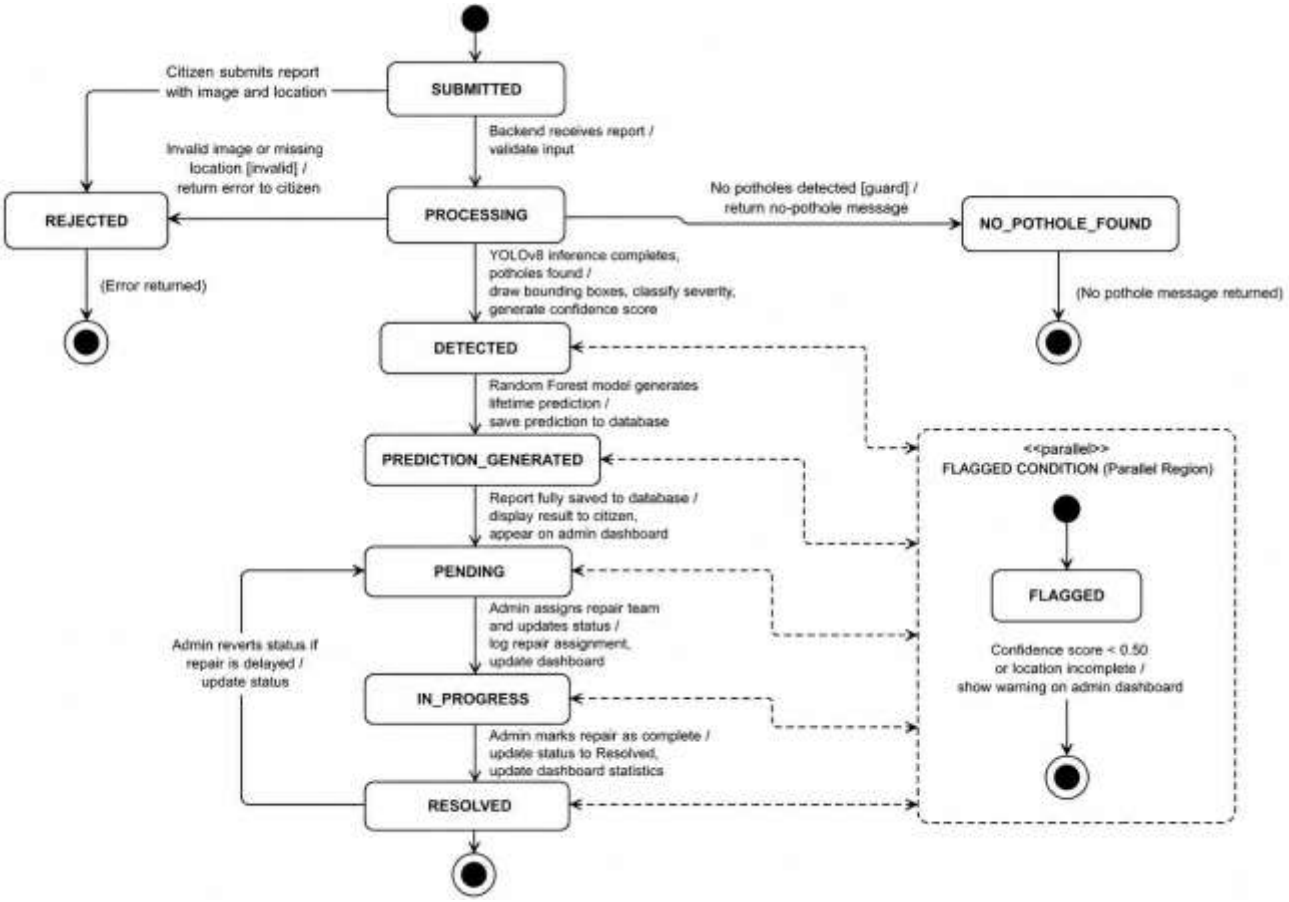


Figure 4.3: State Machine of RahScan System

4.4 Administrative Dashboard and Geographic Heatmap

4.4.1 Description and Priority

This will enable government officials and the road maintenance authority to have a complete web-based control panel that they can use to track, control and take action on all reported instances of road damage. Chengula et al. [8] noted that the current road complaint management systems have deficiencies in their ability to track the repair process, provide any sort of severity-based prioritisation facility and have no streamlined work flow for assigning repairs to specific workers, each of which RahScan addresses. Very significant, this feature is of High Priority.

4.4.2 Stimulus/Response Sequences

- Admin enters the /dashboard login page - System shows the login form.
- If the system recognizes that Admin has entered correct login details then it will be redirected to the main dashboard screen.
- When Admin enters incorrect username/password, the System will output an error message "Incorrect username or password.
- Dashboard loads - All statistics for the overview panel system, the interactive heat map and the report management table are loaded with all those reports that are currently available.
- Admin clicks on a red zone on the heatmap - All heatmap reports in the red zone are highlighted and the report table is scrolled to display the reports relevant to the red zone.
- On the Admin screen, a filter is applied by precedence, city, or date - System maintains an up-to-the-minute report table viewing only the matching reports.
- On the Admin screen, a filter is applied by precedence, city, or date - System maintains an up-to-the-minute report table viewing only the matching reports.
- Admin clicks on a report in the table - System opens up the report detail view containing the annotated image, detection outputs, severity, confidence score, lifetime prediction and current status.
- Admin changes the status of a report - The new status is added to the report table and to the dashboard statistics, with no delay.
- Admin will see the maintenance forecast panel on the system, where the road segments' maintenance forecast for the next 30 days was generated by Facebook Prophet, as proposed by

Taylor and Letham [16] and can be divided into Urgent, Plan Repair and Monitor categories with budget estimates.

4.4.3 Functional Requirements

- **REQ-23:** Restrict access to administrative dashboard to authenticated users (Secure username and password logins).
- **REQ-24:** Present an overall statistics panel in the system to provide information about overall number of received messages, message levels and current rate of resolution, along with monthly and weekly trends.
- **REQ-25:** An interactive Leaflet.js map of the density of the distribution of all reported potholes across Lahore, Karachi and Rawalpindi shall be rendered using the severity distribution as heatmap.
- **REQ-26:** The severity of the high density/severity of the low density area shall be marked on the heatmap by coloring the area red and green respectively.
- **REQ-27:** Systems MUST allow for easy reporting on a city, severity level, date range and repair status.
- **REQ-28:** The system should enable administrator to switch the status of the report in the workflow: Pending - In Progress - Resolved.
- **REQ-29:** The system should be able to record repair team details directly in the Report detail view on the dashboard.
- **REQ-30:** System should present the Predictive Maintenance 30-day forecast on the dashboard, sorted by urgency level and with a prediction of the repair budget.
- **REQ-31:** All current reports and heatmap data should load and display in 5 seconds or less during a normal network conditions.

Sequence Diagram:

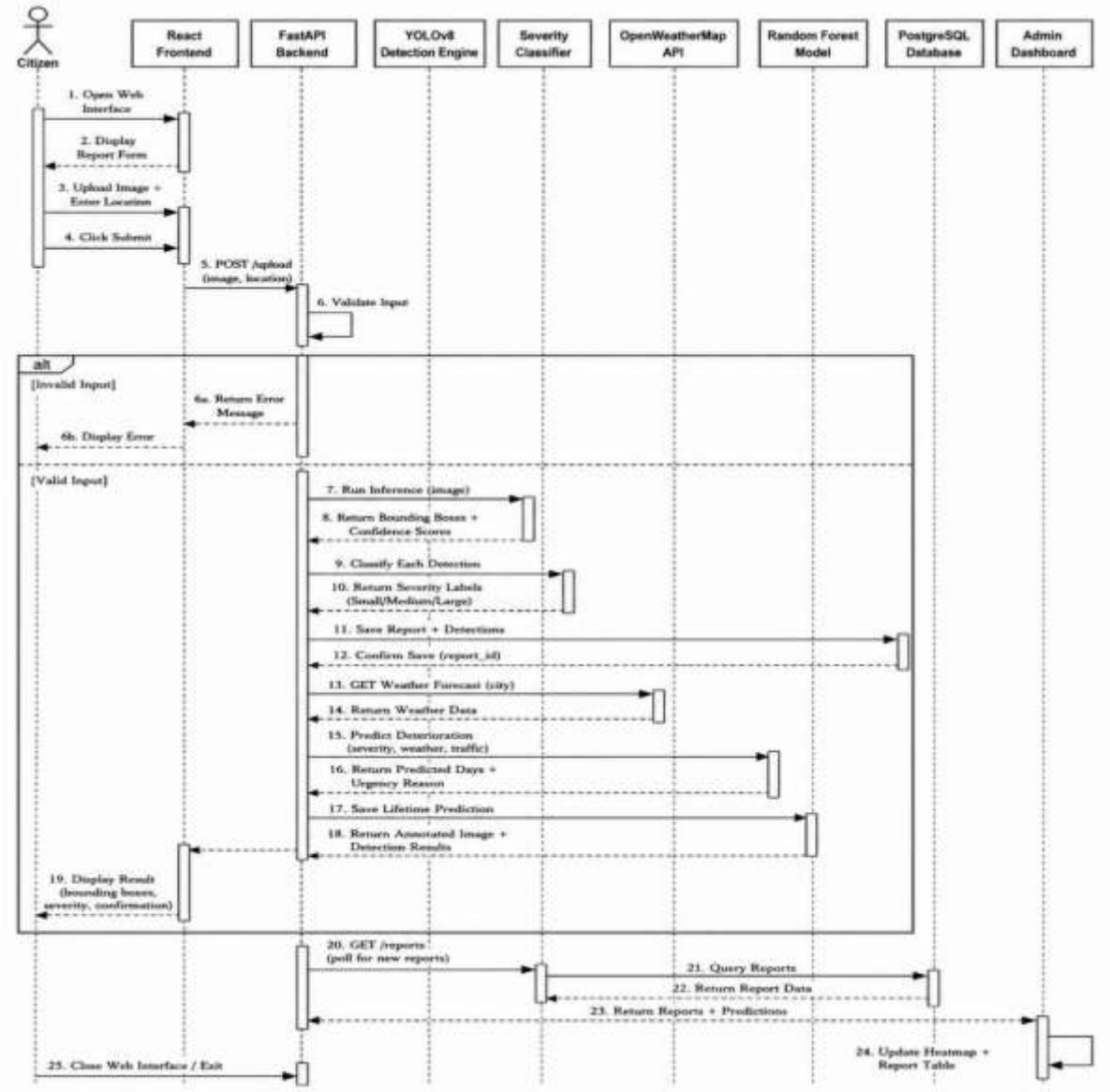


Figure 4.4: Sequence Diagram of RahScan System

4.5 Pothole Lifetime Prediction Module

4.5.1 Description and Priority

This function forecasts the number of days until it becomes a critical potentially dangerous pothole. Temperature, rainfall and means of traffic operational volume are proven by Etak and Soomro [11] to be

most important condition deterioration factors while Random Forest regression models are proven by Marcelino et al. [13] to be most suitable models to predict pavement condition deterioration rates from environmental and means of traffic operational volume data. The Pothole Lifetime Prediction module offered by RahScan implements these findings via a Scikit-learn Random Forest Regression model as described by Pedregosa et al. [18] which uses current visual damage categorization, API traffic density value and API weather forecast data to calculate deterioration time for each identified pothole with a repair deadline and an associated reason for urgency. This functionality is considered to be of High Priority.

4.5.2 Stimulus/Response Sequences

- A new pothole detection report will be stored in the database - For each pothole detected in a report, the system will launch the Lifetime Prediction module.
- The system gets back to us the severity classification of the area, the area area of the bounding box, the type of the road which covered the area and the location from the database. - System sends a request to the OpenWeatherMap API in the form of HTTPS to get the current weather forecast data for the location specified.
- Weather data is obtained successfully - System integrates all the features and feeds them into the trained Random Forest Regression model as described by Pedregosa et al. [18] for inference.
- Model generates prediction - System stores the number of days the model predicted for critical severity, the number of days it is recommended to do the repair, and a reason for urgency in the database.
- Lifetime Prediction Result view Changes made in the Dashboard view - System shows the lifetime prediction result along with the detection outputs in the report detail view.
- If OpenWeatherMap API is not reachable, atmospheric data from the season are used and weather prediction is marked as "NO LIVE DATA".

4.5.3 Functional Requirements

- **REQ-32:** When a pothole is detected the system shall automatically provide a life time deterioration prediction at the time of reporting.
- **REQ-33:** Current severity classification, estimated bounding box area, road type classification, traffic density level and live weather forecast data such as rainfall probability and temperature, and

seasonal indicators are validated features as described by Etak and Soomro [11] and Fakhri and Dezfoulian [12] that are to be used as input to the prediction model.

- **REQ-34:** Implement random forest regression model using Scikit-learn as outlined by Pedregosa et al. [18] for predicting the deterioration.
- **REQ-35:** System to provide the number of days until critical severity for each detection.
- The system should output a recommended repair date for each detected system, and a reason for urgency in language comprehensible to humans.
- **REQ-36:** The result of the life time prediction should be presented on the admin dashboard report detail view for each submitted report.
- **REQ-37:** The system shall gracefully respond to the absence of OpenWeatherMap API by using indicators: seasonal default weather values and marking the affected prediction.

Data Flow Diagram:

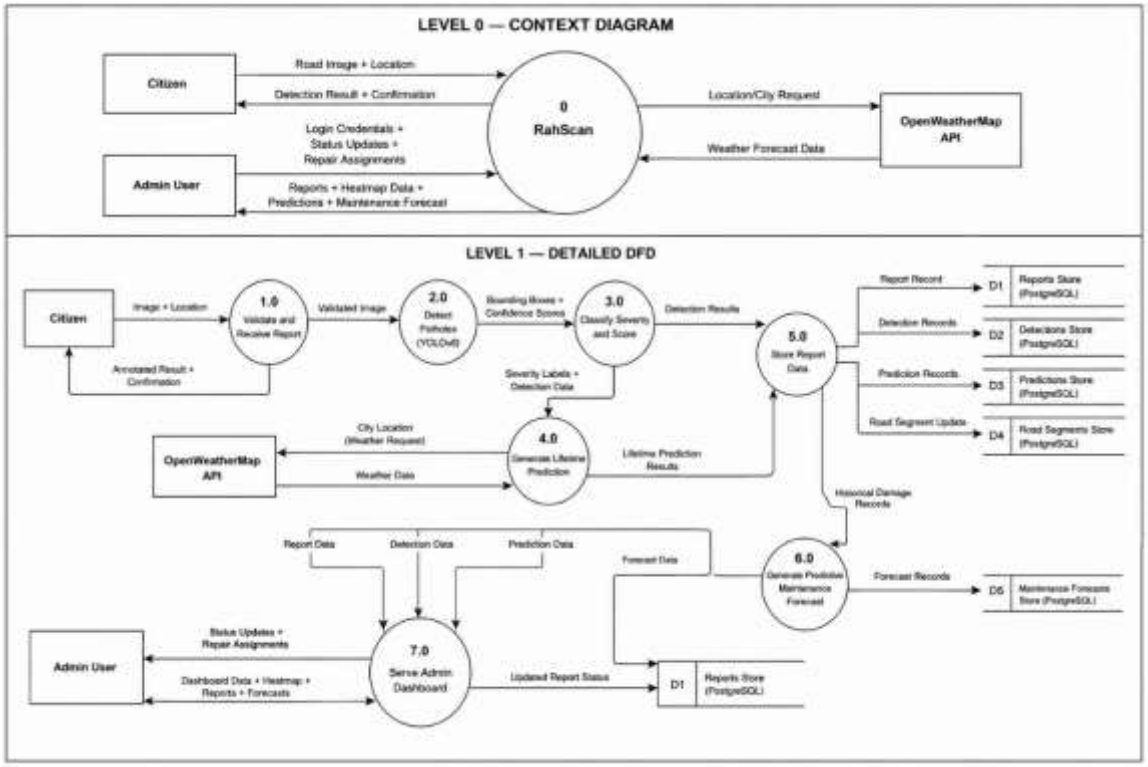


Figure 4.5: Data Flow Diagram of RahScan System

4.6 Predictive Maintenance Module

4.6.1 Description and Priority

It uses the pothole records from the past to uncover roads that regularly develop potholes at consistent times. The review by Matarira et al. [14] identified that time-series forecasting is one of the most effective pavement performance prediction models to detect the pattern of road damage and to produce proactive maintenance plans. It is realized in RahScan via the Facebook Prophet time-series forecasting framework presented by Taylor and Letham in [16] and validated for infrastructure management applications by Daraghmeh et al. in [17] where a 30-day proactive forecast of maintaining the roads is generated with a prioritized repair schedule and estimated budget that prompts authorities to repair infrastructure in advance of accidents. This feature is of **High Priority.**

4.6.2 Stimulus/Response Sequences

- System reads all of the road damage data from the database, categorized by road segment, in the Predictive Maintenance panel on the dashboard.System reads all road damage data from the database, organized by road segment in the Predictive Maintenance panel on the dashboard.
- The historical information is available for a road segment - The time series of damage reports, as defined by Taylor and Letham [16] is passed on to the Facebook Prophet forecasting model.
- Prophet model generates a 30-day forecast: System determines segments of the road network where new pothole occurrence is likely to occur statistically over the next 30 days.
- Each segment is classified and given a priority, those expected in the next 7 days are marked as Urgent; next 8-20 days marked Plan Repair; 20+ days marked Monitor
- System projects estimate repair budget - Budget is estimated based on average repair cost assumption(s) per severity level for the number of potholes that are anticipated.
- Complete forecast is displayed in system and priority maintenance schedule with urgency labeling & budget estimates on the Admin dashboard.
- Lack of historical data – uses synthetic historical data, and marks it as such on the dashboard output.

4.6.3 Functional Requirements

- **REQ-39:** The system will analyze logs of previous damage to the road network stored in the PostgreSQL database to determine if there are patterns of damage on certain sections of the road.
- **REQ-40:** The system shall use the time series forecasting library “Prophet” introduced by Taylor and Letham [16] for Facebook to forecast damages on road segments.
- **REQ-41:** Generate 30 day forward maintenance forecast, identifying road segments where, statistically, new potholes are likely to occur within the 30 days forecast.
- **REQ-42:** The system should classify regions of the roadway being modelled into an Urgent, Plan Repair or Monitor category, according to the time until the next expected pothole event.
- **REQ-43:** The system should calculate an estimated repair budget for the 30-day prediction horizon based on the number and severity of potholes to be expected along a road.
- **REQ-44:** The System shall present the full 30-day maintenance forecast/budget in the administrative Dashboard in a dedicated Predictive Maintenance panel.
- **REQ-45:** All the forecasts generated from the synthetic historical data shall be clearly marked as synthetic in the dashboard display, according to the transparency principle recommended by Matarira et al. [14] for EEPMSs.

Component Diagram:

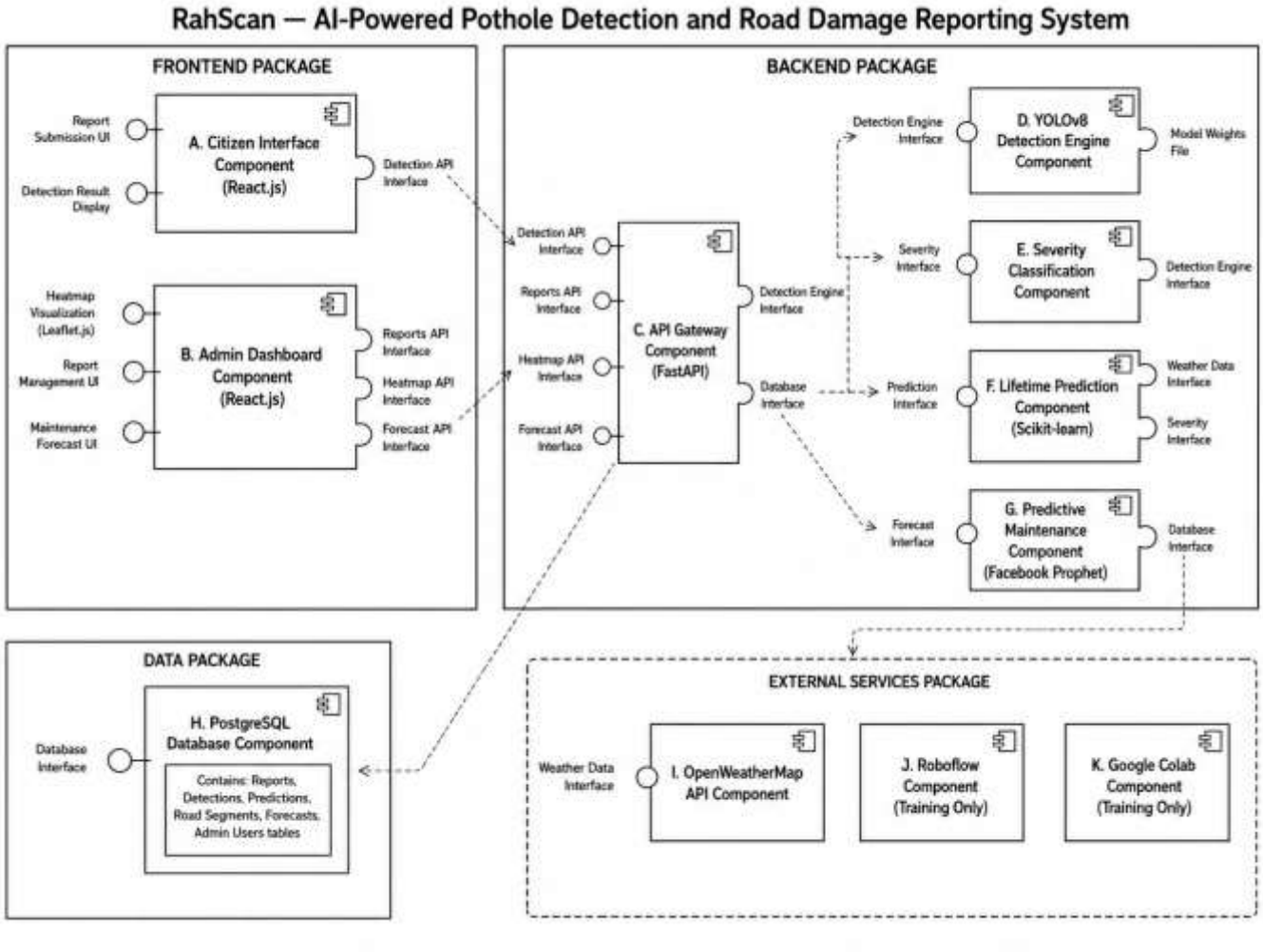


Figure 4.6: Component Diagram of RahScan System

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Performance is the key to RahScan as citizens and administrators demand quick and responsive results. The system should meet the following requirements:

- **Detection Inference Speed** Under the normal server load, the YOLOv8 pothole detection engine developed by the Jocher et al. [5] shall be able to detect and annotate the results of a previously

submitted image within 5 seconds. This guarantees citizens timely feedback following the submission of a report, without having a frustrating journey down the long delay/queue..

- **Dashboard Load Time** Within 5 seconds of a standard broadband connection, the whole admin dashboard, including the geographic heat map and dashboard report management table shall be fully loaded and display all existing data.
- **API Response Time** All RESTful API endpoints will respond to basic data retrieval operations (e.g., reports, trends, coordinates, heatmap, etc.) within 3 seconds of the request under normal server load.
- **Concurrent Users** The system shall be capable of handling a minimum of 50 citizen users reporting concurrently and in parallel, without compromise to system stability or response time.
- **Database Query Performance** All database queries that are used for report retrieval, filtering, or aggregation should be completed within 2 seconds for datasets of up to 10,000 stored reports.
- **Lifetime Prediction Generation** Lifetime Prediction Generation: Pothole Lifetime Prediction module shall deliver a pothole deterioration forecast within 2 seconds of detecting potholes including the call to the weather API and model inference.
- **Predictive Maintenance Forecast Generation** The complete 30 day forecast for maintenance shall be generated or returned within 10 seconds after being requested from the Admin Dashboard.
- **Image Upload Size** Resize of images up to 10 Mb shall be possible without problems of performance or timeouts.

5.2 Safety Requirements

RahScan is a piece of road safety technology, and as such, it must work properly and not worsen road safety. The following safety rules need to be adhered:

- **No False Safety Assurance** The system will never give any indication of safety for a road or that there is no pothole, on the basis of just one image that the system has been fed. Individual reports should not be used as a basis for conclusions about road safety.
- **Low Confidence Flagging** If the YOLOv8 model (Jocher et al. [5]) confidence score is below 0.50, the detection result will automatically be marked as a low-confidence detection in the database and on the admin dashboard and the admin will be notified to manually check the report for accuracy before sending a repair party.

- **Incomplete Location Flagging** Reports submitted with incomplete, unclear or incomplete location data shall be automatically marked in the database as potential data quality concerns and will be sent to administrators so they know if there are any problems in the data.
- **Data Loss Prevention** All the submitted reports and detection results shall be immediately persisted into the PostgreSQL database after successful processing. Report data is not lost in case of timeout or shut down of the 'frontend' session (browser) after successful processing of the 'backend' session.
- **Graceful Degradation** If the OpenWeatherMap API is not available, the Pothole Lifetime Prediction module shall not crash or stall the detection pipeline. It shall revert to seasonal default weather values, fill in the prediction and set the generated prediction to No live weather data
- **No Harmful Recommendations** The system shall not make repair budget and/or repair scheduling recommendations that may result in intentionally unsafe road conditions. All Urgent level forecasts in the Predictive Maintenance module will continue to be active on the admin dashboard until they are manually addressed by an admin.

5.3 Security Requirements

RahScan reports and records road damage, location information and provides administrative decision-making tools. The following security requirements apply:

- **Admin Authentication** Administrative Dashboard Access to Administrative Dashboard shall be secured by a secure login system with a valid username and password. Allow only authenticated users access to admin only features, data and/or API endpoints.
- **Role-Based Access Control** Only Citizen users who match the role in the report will be able to submit reports and see their own detection results. Administrative users are allowed access to all reports, the heatmap, the prediction outputs, the status management and the maintenance forecast. No citizen user will have access to or ability to alter admin data.
- **Input Validation** All input including images uploaded by the user, localizumab and form fields shall be checked on the backend before it is processed. Any file received that is suspected of being malicious (attached to an image) will be rejected. Prevention of SQL injection attacks and cross-site scripting (XSS) shall be done via parameterization of database queries and proper sanitization of database inputs.

- **Secure API Communication** Production deployment of API Communication Secure All API communication communicate between the frontend and the FastAPI backend shall be made with HTTPS to safeguard the data in transit from being intercepted.
- **Database Security** PostgreSQL database should not be accessible over the public Internet. It should only be available via local connect from the back end server. All database credentials must be kept in environment variables and not in the source code.
- **API Key Protection** OpenWeatherMap API key shall be saved appropriately as server-side environment variable it will never be made visible on the client application (frontend). It shall not be available publicly in any public repository.
- **Session Management** Admin sessions for the Session Management will end after 30 minutes of inactivity. If session expires, admin will automatically be logged off and redirected to the login page.
- **Password Storage** Admin passwords will be stored in the database as a securely hashed value, using a recognized hashing algorithm. There shall never be a storage of plain text passwords.

5.4 Software Quality Attributes

RahScan's reliable, maintainable and usable in real-time municipal scenario in Pakistan require the following quality attributes:

- **Reliability:** The system will provide an uptime of 95 percent or greater during the demonstration and evaluation period. All API endpoints shall have error handling on the backend to prevent the system from crashing if there is a failure within one module. Any critical error found will be recorded for troubleshooting.
- **Usability**: Usability The citizen reporting interface shall be usable without needing to provide any instructions or assisting the citizen in that task. All labels, buttons and error messages need to be written in everyday easy-to-understand English. The admin dashboard should have a consistent layout, a clean interface and labelled panels and controls.
- **Maintainability**: All back-end code will be used in clearly separated modules for detection, severity classification, prediction, database operation and API routing. Comments in the code should be sufficient. The database structure will be able to expand to accommodate new (report) fields or modules without the need for a complete restructuring.

- **Scalability:** The Scalability System architecture will be designed to scale to a larger number of reports over time without major restructuring. This is a must-have feature as the database is scaled to store reports larger than 10,000, and require an efficient use of the database for queries.When the number of reports will exceed 10,000, the PostgreSQL database schema and API design shall support pagination and efficient queries.
- **Portability:** Portability RahScan to be run on any standard Linux server running Ubuntu 22.04 or above with any version of Python 3.10 or above. All dependencies will be recorded in a requirements file with automatic configuration of the system on a new server.
- **Testability:** All back-end modules such as the detection pipeline, severity classification layer, prediction engine and all of the API endpoints shall be designed to be testable. All unit tests will be done with regular python testing modules and will run from command line.
- **Correctness:** The system shall always generate Detection, Severity Classification and Prediction output results that are in line with the trained model and the logic programming. The output of no module shall be inconsistent with the input received into it.
- **Availability**: Raw error messages or the stack trace should not be shown to an end user. All error scenarios shall be accompanied by user friendly error pages and/or errors.

5.5 Business Rules

The following rules govern how RahScan works:

- **BR-1:** Only people who work with RahScan and have the access can change the status of a repair report. Regular people can send in reports. See the results but they cannot change anything that has already been saved.
- **BR-2:** A report can only move forward from Pending to In Progress to Resolved. Once a report is marked as Resolved it will not be reopened by the system. If someone sends in a report about the same road it will be treated as a new report.
- **BR-3:** Every picture that is sent in must have a location attached to it. It will not be accepted. If a report does not have a location it will be rejected away.
- **BR-4:** The Pothole Lifetime Prediction part of RahScan will make a prediction for every pothole that is found no matter how bad it is. No pothole will be left out just because it is small.

- **BR-5:** The Predictive Maintenance part of RahScan will only make forecasts for road segments that have least two records of damage in the database. If a road segment only has one record it will say that there is not data to make a forecast.
- **BR-6:** If RahScan is not very sure about a detection and its confidence score is below 0.50 it will still be. Shown to administrators but it will have a warning sign so they know to be careful when making decisions about it.
- **BR-7:** RahScan will not automatically send out repair teams or notifications to systems. Administrators have to do these things through the dashboard.
- **BR-8:** During the stages of RahScan some data will be made up to help the Predictive Maintenance part work better. This made-up data will be clearly labeled in the dashboard so administrators know it is not real and do not use it to make decisions, about repairs.

5.6 Detailed Subsystem Design

RahScan is composed of six interconnected subsystems the Citizen Reporting Interface, the AI Detection Engine, the Severity Classification and Confidence Scoring Layer, the Administrative Dashboard and Heatmap, the Pothole Lifetime Prediction Module, and the Predictive Maintenance Module. Each subsystem handles a distinct phase of the road damage detection, reporting, and maintenance planning pipeline, and all subsystems communicate through the central FastAPI backend and PostgreSQL database.

The following diagrams shall be produced to fully document the structure, behavior, and data flow of RahScan:

- **ER Diagram** Showing the complete relational database schema including all tables, attributes, primary keys, foreign keys, and relationships.

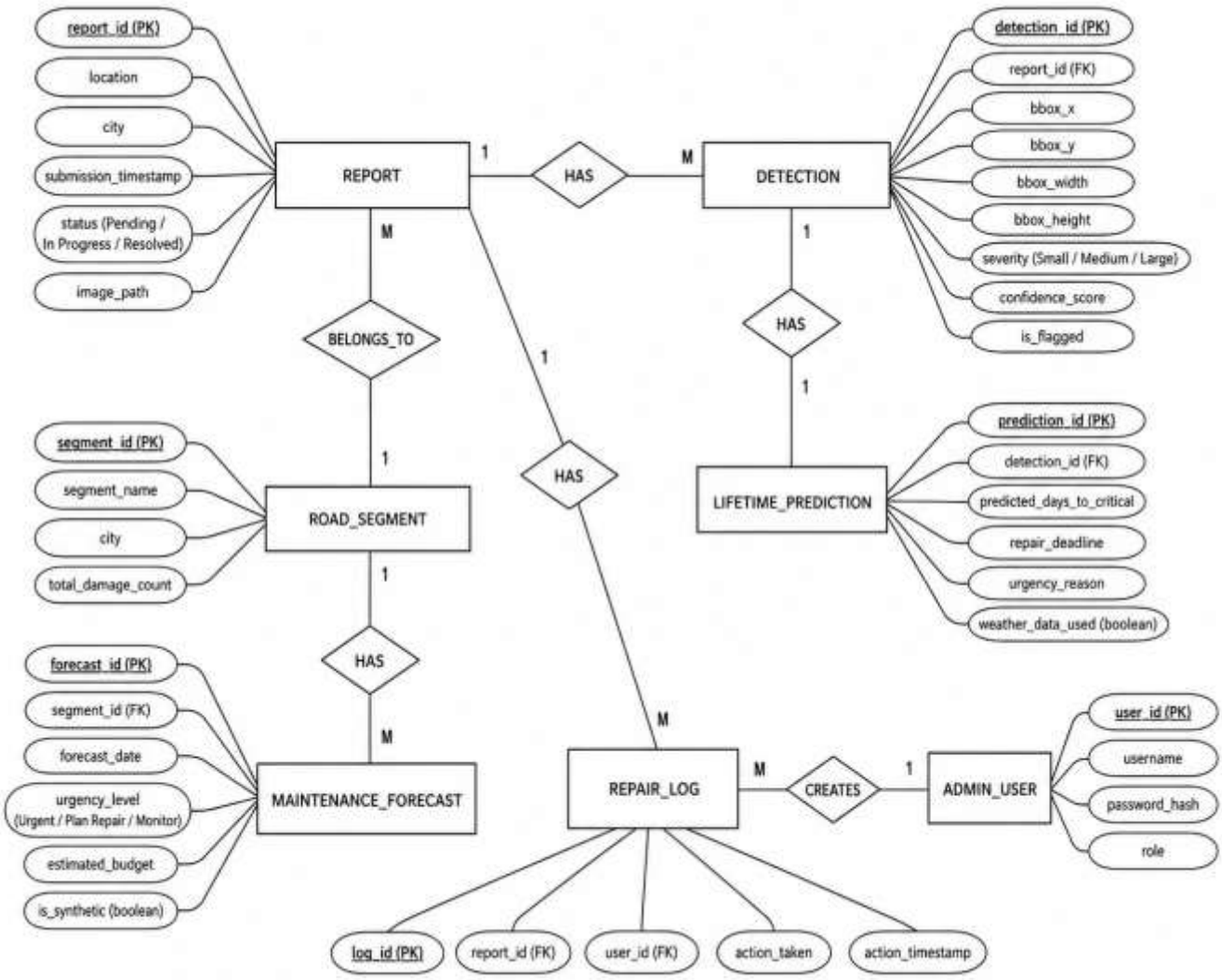


Figure 5.6.1: ER Diagram of RahScan System

- **Architectural Diagram** Showing the high-level three-layer architecture of RahScan including the citizen interface, FastAPI backend, AI detection pipeline, prediction modules, PostgreSQL database, and admin dashboard.

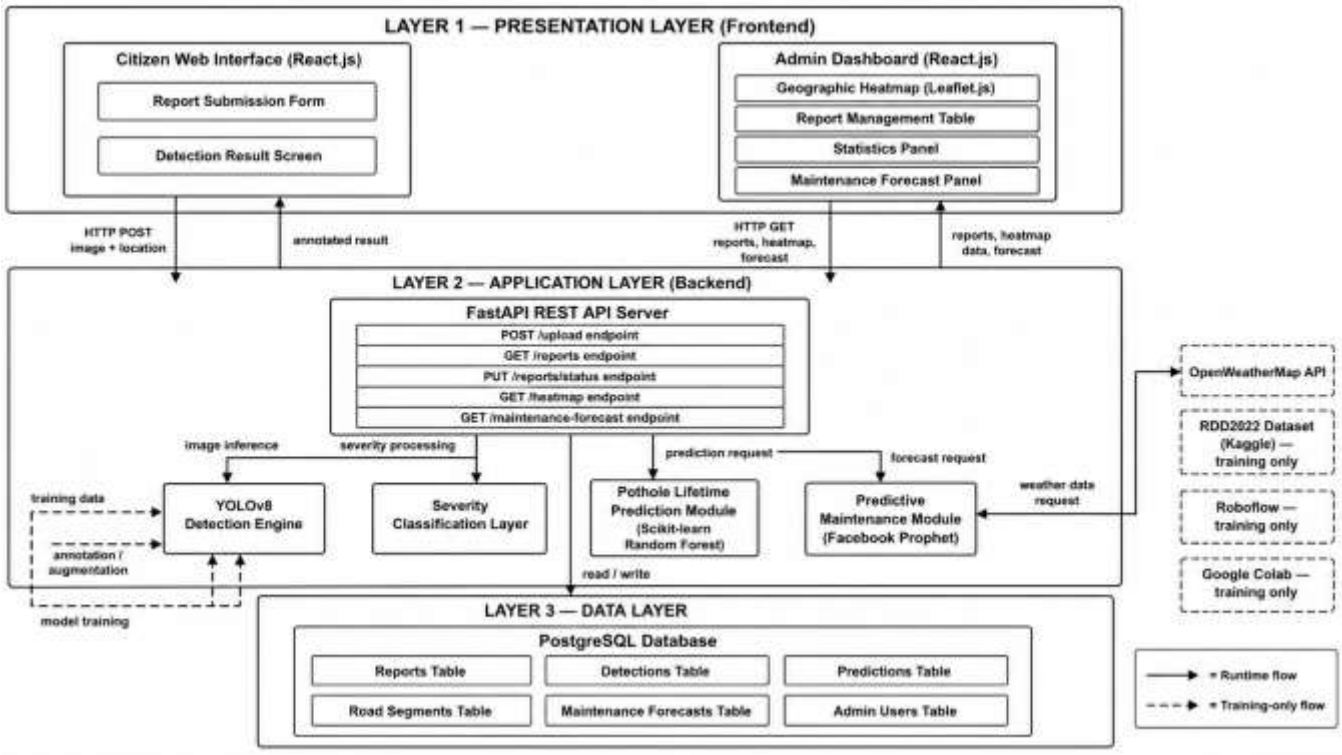


Figure 5.6: Architectural Diagram of RahScan System

- **Class Diagram** Showing the object-oriented structure of the RahScan backend including all classes, attributes, methods, and relationships between the Report, Detection, Prediction, MaintenanceForecast, and User entities.

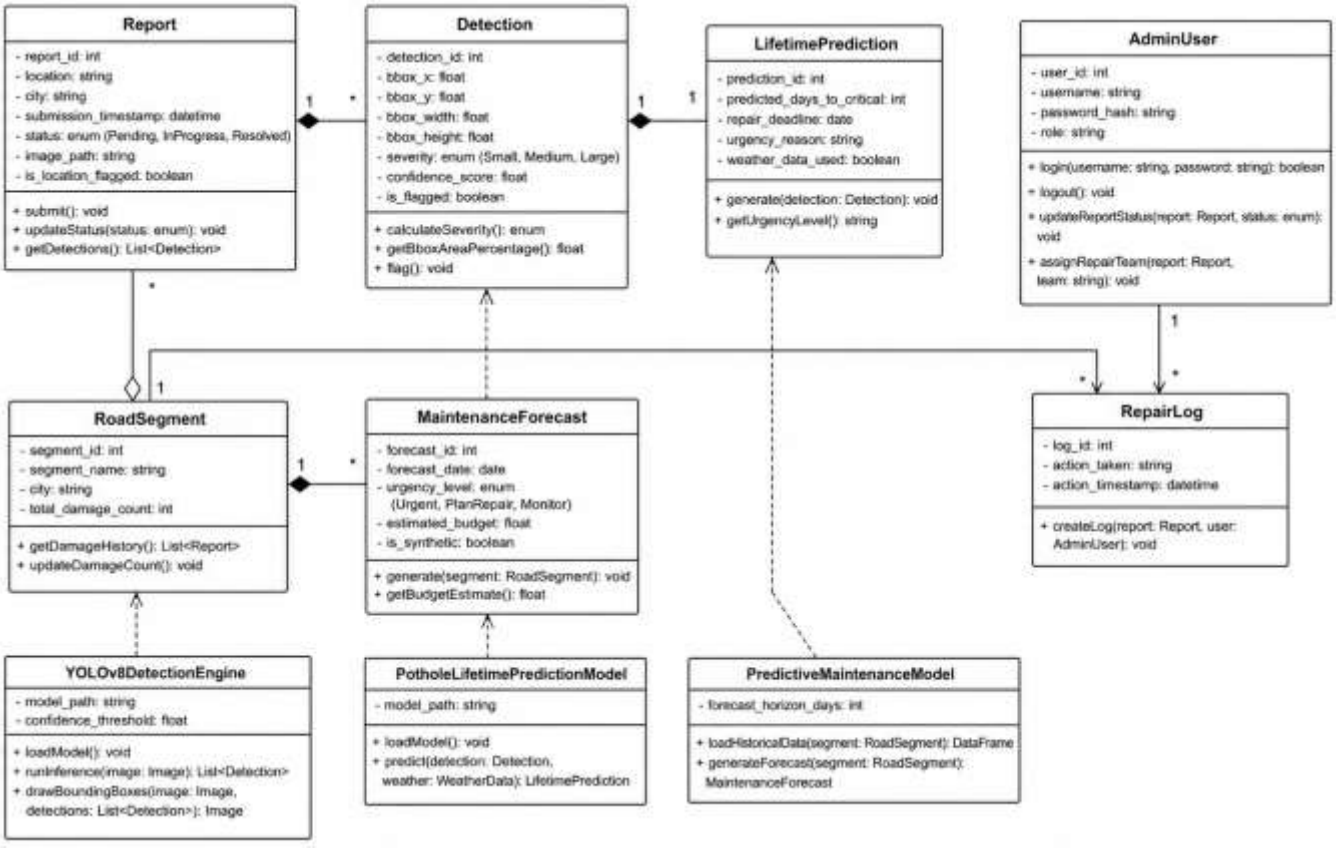


Figure 5.6.2: Class Diagram of RahScan System

- **Database Diagram** Showing the complete physical database design including all tables, columns, data types, constraints, and foreign key relationships in the PostgreSQL schema.

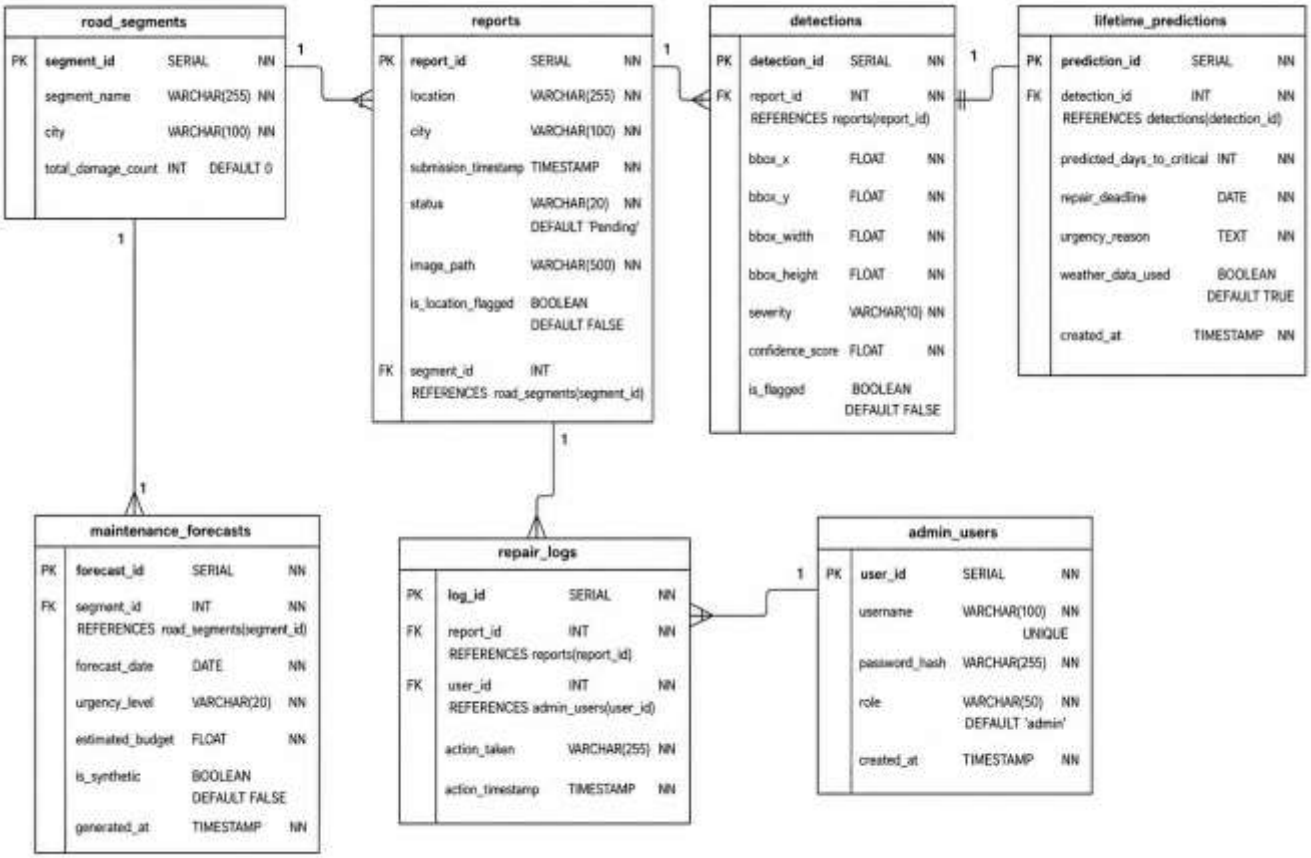


Figure 5.6.3: Database Diagram of RahScan System

REFERENCES

[1] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, "Pothole detection using deep learning: a real-time and AI-on-the-edge perspective," *Advances in Civil Engineering*, vol. 2022, pp. 1–13, Apr. 2022, doi: 10.1155/2022/9221211.
[Online]. Available: https://onlinelibrary.wiley.com/doi/10.1155/2022/9221211

[2] N. K. Rout, G. Dutta, V. Sinha, A. Dey, S. Mukherjee, and G. Gupta, "Improved pothole detection using YOLOv7 and ESRGAN," *arXiv preprint*, arXiv:2401.08588, Dec. 2023.
[Online]. Available: https://arxiv.org/abs/2401.08588

[3] R. Thakur, Bhumika, P. Kumar, and P. Thakur, "YOLOv8-based pothole detection: a real-time approach for road infrastructure monitoring," *SSRN Electronic Journal*, Nov. 2024, doi: 10.2139/ssrn.5089189. [Online]. Available: https://ssrn.com/abstract=5089189

[4] Bhavana N. et al., "POT-YOLO: real-time road potholes detection using edge segmentation based YOLOv8 network," *IEEE Sensors Journal*, vol. 24, no. 13, 2024, doi: 10.1109/JSEN.2024.3399215.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, version 8.0.0, Jan. 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2022: a multi-national image dataset for automatic road damage detection," *Geoscience Data Journal*, vol. 11, pp. 846–862, Jun. 2024, doi: 10.1002/gdj3.260 [Online]. Available: https://rmets.onlinelibrary.wiley.com/doi/10.1002/gdj3.260

[7] E. Salcedo, M. Jaber, and J. Requena Carrion, "A novel road maintenance prioritisation system based on computer vision and crowdsourced reporting," *Journal of Sensor and Actuator Networks*, vol. 11, no. 1, p. 15, Feb. 2022, doi: 10.3390/jsan11010015.
[Online]. Available: https://www.mdpi.com/2224-2708/11/1/15

[8] J. Chengula, J. Mwanza, N. Lucas, and D. Mwanza, "Augmenting roadway safety with machine learning and deep learning: pothole detection and dimension estimation using in-vehicle technologies," *Machine Learning with Applications*, vol. 16, p. 100547, Jun. 2024, doi: 10.1016/j.mlwa.2024.100547. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666827024000239

[9] L. Li, J. Liu, J. Xing, Z. Liu, K. Lin, and B. Du, "Road pothole detection based on crowdsourced data and extended Mask R-CNN," *IEEE Transactions on Intelligent Transportation Systems*, early access, 2024, doi: 10.1109/TITS.2024.3367891.

[10] S. Swami and R. Jangid, "Pothole detection and prediction using ResNet50 and YOLOv8," in *Lecture Notes in Networks and Systems*, Springer, 2024.

[11] R. Etak and N. Soomro, "Prediction and detection of potholes in urban roads: machine learning and deep learning based image segmentation approaches," *Development Engineering*, vol. 8, p. 100100, 2022, doi: 10.1016/j.deveng.2022.100100.

[12] A. Fakhri and S. Dezfoulian, "Climate change impact on infrastructure: a machine learning solution for predicting pavement condition index," *Construction and Building Materials*, vol. 315, p. 125658, Jan. 2022, doi: 10.1016/j.conbuildmat.2021.125658.

[13] D. M. Marcelino, M. de Lurdes Antunes, E. Fortunato, and M. Gomes, "Machine learning algorithms for monitoring pavement performance," *Automation in Construction*, vol. 136, p. 104190, Apr. 2022, doi: 10.1016/j.autcon.2022.104190.

[14] T. Matarira, G. Mutanga, N. Naicker, and M. Mushore, "Pavement performance model for road maintenance and repair planning: a review of predictive techniques," *Digital Transportation and Safety*, vol. 2, no. 4, pp. 253–267, Dec. 2023, doi: 10.48130/DTS-2023-0021. [Online]. Available: https://doi.org/10.48130/DTS-2023-0021

[15] A. Karballaeezadeh, S. Zaremotekhases, S. Shamshirband, A. Mosavi, P. Nabipour, P. Csiba, and A. R. Varkonyi-Koczy, "Intelligent road inspection with advanced machine learning: hybrid prediction models for smart mobility and transportation maintenance systems," *Energies*, vol. 13, no. 8, p. 1919, 2020, doi: 10.3390/en13081919.

[16] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, Apr. 2018, doi: 10.1080/00031305.2017.1380080.
[Online]. Available: https://www.tandfonline.com/doi/full/10.1080/00031305.2017.1380080

[17] M. Daraghmeh, S. Al-Qadeeb, and A. Al-Zinati, "Time series forecasting using Facebook Prophet for cloud resource management," in *Proc. IEEE Int. Conf. on Innovations in Intelligent Systems and Applications (INISTA)*, 2021, doi: 10.1109/INISTA52262.2021.9473607.
[Online]. Available: https://ieeexplore.ieee.org/document/9473607

[18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html